SE 410

Software Framework Applications

Asst. Prof. Serhat Uzunbayır

Project Report

E-Learning Platform

Group 7

Defne Yılmaz

Hasan Efe Ünal

Can Özer

Ege Yılmaz

Contents

1. Introduction
   1.1. Project Overview and Objectives
   1.2. System Architecture
   1.3. Technology Stack
   1.4. Improvements on V1
2. Requirements
   2.1. Functional Requirements
   2.2. Non-Functional Requirements
3. Database Integration
   3.1. Database Schema
   3.2. Entity Framework Core
4. LINQ Queries
   4.1. Course catalogue filtering (Where, OrderBy, Select)
   4.2. Enrolment analytics (GroupBy, Count)
   4.3. Progress calculation (Join, Average)
   4.4. Study plan generation (OrderBy, ThenByDescending)
   4.5. Course recommendations (Where, Any, Take)
5. Delegates and Events
   5.1. EnrolmentService
   5.2. StudyPlanService
   5.3. CourseRecommendationService
   5.4. Design benefit
6. MVC Structure
7. Security
   7.1. Password hashing (BCrypt)
   7.2. JWT authentication
   7.3. Role-based authorization
   7.4. Two-factor authentication (2FA)

7.5. Input validation
8. Sources
9. User Interface Design
9.1. Web application (MVC)
9.2. Desktop application (WinForms)
10. Tests
11. Conclusion

1. Introduction

1.1. Project Overview and Objectives

The "E-Learning Platform with Personalized Study Plans" is a multi-client learning management system that lets students enrol in courses, complete course modules at their own pace, and receive an automatically generated, ECTS-balanced weekly study plan. Administrators manage the course catalogue, approve enrolment requests, monitor student progress, and analyse course-level metrics.

The main objective of the project is to demonstrate a realistic, framework-based software application built on the .NET ecosystem, combining three different application styles around a single shared database:

- REST API that owns all business logic and data access

- Web application built with the ASP.NET Core MVC pattern

- Desktop application built with Windows Forms

1.2. System Architecture

The solution "learning_platform.sln" is composed of four projects:

- LearningPlatform.API (ASP.NET Core Web API) — models, EF Core data layer, business logic, JWT/2FA, controllers

- Learning_platform (ASP.NET Core MVC) — student-facing web interface

- LearningPlatform.Desktop (Windows Forms) — administrator and student desktop interface

- LearningPlatform.API.Tests (xUnit) — integration tests

A typical end-to-end flow is:

- A student logs in through the web app, browses and filters the course catalogue, and sends an enrolment request.

- An administrator reviews the request from the desktop app and approves it; the API raises a delegate-based notification event.

- The student accesses course modules and marks them as completed; the API tracks the completion percentage.

- The student generates a personalised study plan; the API orders courses by completion ratio and ECTS load.

- The system recommends new courses through a delegate, based on the student's preferred category and history.

1.3. Technology Stack

- .NET / C# — unified platform for API, web, and desktop

- ASP.NET Core Web API — REST endpoints, dependency injection, authentication
- ASP.NET Core MVC — server-rendered student UI
- Windows Forms — native desktop admin tool
- Entity Framework Core — code-first ORM with migrations
- SQLite — portable file-based database (learning_platform.db)
- JWT Bearer — stateless role-based authentication
- BCrypt.Net — password hashing
- xUnit — integration testing

1.4. Improvements on V1

In version 1, the team delivered the REST API, the student web application, and the basic desktop structure, covering registration/login with JWT, BCrypt, course CRUD, enrolment approval, module completion, progress tracking, study-plan generation, and the LINQ/delegate requirements.

Version 2 completes the remaining application requirements:

- Two-factor authentication: 6-digit, time-limited 2FA on login
- Admin analytics: enrolment numbers, completion rates, activity metrics
- Course module management: full module CRUD for administrators
- Course recommendations: personalised recommendations via delegate/event
- Database: relative-path resolution so the app runs on any machine unchanged
- Automated tests: 11 xUnit integration tests (all passing)
- User interface: modernised web and desktop UI

2. Requirements

2.1. Functional Requirements

- Administrators can create courses (title, description, category, difficulty, ECTS)
- Administrators can edit or delete existing courses
- Students can browse and view all available courses on the web
- Students request enrolment; administrators must approve before access
- Students access course modules and mark them as completed
- The system tracks progress and shows the completion percentage
- The system generates personalised study plans (progress + ECTS)
- The system recommends courses using delegates
- Administrators can view and monitor student progress on the desktop app

- Administrators analyse course data (enrolments, completion rates, activity)

All functional requirements were completed successfully.

2.2. Non-Functional Requirements

- Consistent, intuitive UI with clear navigation: shared layout on the web, sidebar shell on desktop, success/error feedback on every action

- Secure data handling: BCrypt password hashing, JWT signed with a secret key

- Two-factor authentication: 5-minute code verified before the JWT is issued

- Shared SQLite database, consistent across clients; all access through the API

- Documentation: this report and a user manual (see Section 8)

3. Database Integration

3.1. Database Schema

The database has seven tables, managed with code-first Entity Framework Core. The runtime file is "learning_platform.db" at the repository root, shared by every client through the API.

1. Users — students and administrators (Role column); BCrypt password hashes; 2FA fields; preferred category

2. Courses — catalogue entries (title, description, category, difficulty, ECTS)

3. CourseModules — ordered learning units belonging to a course

4. Enrolments — student requests (Pending, Approved, Rejected)

5. ModuleCompletions — one row per completed module per student

6. StudyPlans — one plan per student

7. StudyPlanItems — scheduled course inside a plan (day + recommended hours)

3.2. Entity Framework Core

EF Core DbContext (LearningPlatformContext) exposes DbSets for all entities. Migrations are applied at API startup. Navigation properties link Courses to CourseModules, Enrolments to Users and Courses, and StudyPlans to StudyPlanItems. No client application opens the database directly; the API is the single data access layer, which guarantees consistency between the MVC web app and the WinForms desktop app.

4. LINQ Queries

LINQ (Language-Integrated Query) is used throughout the API for filtering, sorting, grouping, projection, and aggregation. Representative examples:

4.1. Course catalogue filtering (Where, OrderBy, Select)

Students filter courses by category and difficulty on the web catalogue:

```
var query = _context.Courses.AsQueryable();
if (!string.IsNullOrEmpty(category))
    query = query.Where(c => c.Category == category);
if (maxDifficulty.HasValue)
    query = query.Where(c => (int)c.Difficulty <= maxDifficulty.Value);
var courses = await query
    .OrderBy(c => c.Title)
    .Select(c => new CourseListDto {
        Id = c.Id, Title = c.Title, Category = c.Category,
        Difficulty = c.Difficulty, Ects = c.Ects
    })
    .ToListAsync();
```

This query composes filters at runtime and projects entities to DTOs for the API response.

4.2. Enrolment analytics (GroupBy, Count)

Administrators view enrolment counts per course on the desktop analytics screen:

```
var enrolmentStats = await _context.Enrolments
    .Where(e => e.Status == EnrolmentStatus.Approved)
    .GroupBy(e => e.CourseId)
    .Select(g => new CourseEnrolmentStatDto {
        CourseId = g.Key,
        EnrolmentCount = g.Count()
    })
    .OrderByDescending(x => x.EnrolmentCount)
    .ToListAsync();
```

4.3. Progress calculation (Join, Average)

Completion percentage joins modules, completions, and approved enrolments:

```
var progress = await (
    from e in _context.Enrolments
    join c in _context.Courses on e.CourseId equals c.Id
    where e.StudentId == studentId && e.Status == EnrolmentStatus.Approved
    let totalModules = c.Modules.Count()
    let completed = _context.ModuleCompletions.Count(
        mc => mc.StudentId == studentId && mc.Module.CourseId == c.Id)
    select new ProgressDto {
        CourseId = c.Id,
        Title = c.Title,
        CompletionPercent = totalModules == 0 ? 0 : (completed * 100) /
```

```
totalModules
    }).ToListAsync();
```

4.4. Study plan generation (OrderBy, ThenByDescending)

When a student generates a plan, courses are prioritised by lowest completion first, then by ECTS load:

```
var ranked = enrolledCourses
    .Select(c => new {
        Course = c,
        Ratio = c.TotalModules == 0 ? 0.0 :
            (double)c.CompletedModules / c.TotalModules
    })
    .OrderBy(x => x.Ratio)
    .ThenByDescending(x => x.Course.Ects)
    .ToList();
```

4.5. Course recommendations (Where, Any, Take)

Recommendations exclude already enrolled or completed courses and prefer the student's category:

```
var enrolledIds = await _context.Enrolments
    .Where(e => e.StudentId == studentId)
    .Select(e => e.CourseId)
    .ToListAsync();
var recommendations = await _context.Courses
    .Where(c => c.Category == preferredCategory && !enrolledIds.Contains(c.Id))
    .OrderByDescending(c => c.Ects)
    .Take(5)
    .Select(c => new CourseListDto { Id = c.Id, Title = c.Title })
    .ToListAsync();
```

5. Delegates and Events

The project uses C# delegates and events to decouple "something important happened" from "how the system reacts." Three service classes each declare a delegate type, expose an event, subscribe a default handler in the constructor, and raise the event after a state change. Controllers call the service method without knowing who listens.

5.1. EnrolmentService

```
public delegate void EnrolmentApprovedHandler(
    int enrolmentId, int studentId, int courseId, string courseTitle);
public event EnrolmentApprovedHandler? EnrolmentApproved;

public void ApproveEnrolment(int enrolmentId) {
    // ... update database ...
    OnEnrolmentApproved(enrolmentId, studentId, courseId, course.Title);
}
protected virtual void OnEnrolmentApproved(...) =>
    EnrolmentApproved?.Invoke(...);
```

Default handler logs the approval and can be extended later for email notifications without changing the controller.

5.2. StudyPlanService

```
public delegate void StudyPlanGeneratedHandler(int studentId, int itemCount);
public event StudyPlanGeneratedHandler? StudyPlanGenerated;
```

Raised after a new study plan is persisted so the desktop admin dashboard can refresh activity metrics.

5.3. CourseRecommendationService

```
public delegate void CoursesRecommendedHandler(
    int studentId, IReadOnlyList<int> courseIds);
public event CoursesRecommendedHandler? CoursesRecommended;
```

Raised when the recommendation engine returns results, allowing the MVC layer to show a toast or highlight suggested courses.

5.4. Design benefit

This pattern satisfies the course requirement for delegates/events while keeping controllers thin. New subscribers (logging, analytics, future SignalR hub) can be added without modifying existing approval or plan-generation logic.

6. MVC Structure

The student-facing web application uses ASP.NET Core MVC with three layers:

- Model — DTO classes (e.g. CourseDtos, EnrollmentDtos) exchanged with the API

- View — Razor .cshtml files; HTML only, no business logic

- Controller — HTTP handling, API calls via injected services, view selection

Controllers and responsibilities:

1. AuthController — login, 2FA verification, JWT stored in auth cookie

2. HomeController — landing page and redirects

3. CoursesController — browse/filter, details, enrolment request

4. DashboardController — enrolled courses and overall progress

5. ProgressController — module list and mark complete

6. StudyPlanController — view and generate study plan

7. AdminController — course and module CRUD in the browser

7. Security

7.1. Password hashing (BCrypt)

Passwords are never stored in plain text. On registration, BCrypt.Net generates a salted hash stored in Users.PasswordHash. Login verifies with BCrypt.Verify. Even if the database file is copied, recovered passwords remain impractical to obtain.

7.2. JWT authentication

After successful login and 2FA, the API issues a signed JWT containing UserId, email, and role (Student or Admin). Clients send Authorization: Bearer <token>. The API validates issuer, audience, signature, and expiry on every protected endpoint. The MVC app stores the token in an HTTP-only cookie for convenience.

7.3. Role-based authorization

Endpoints and controllers use [Authorize(Roles = "Admin")] or [Authorize(Roles = "Student")] so students cannot approve enrolments or delete courses, and guests cannot access protected resources. The desktop app sends the same JWT obtained at login.

7.4. Two-factor authentication (2FA)

When 2FA is enabled for a user, the login endpoint validates email/password first, then generates a random six-digit code stored with a five-minute expiry. The client must call /auth/verify-2fa with the correct code before receiving a JWT. This adds a second factor even though codes are currently displayed in the API response for demo purposes (production would use email/SMS).

7.5. Input validation

DTOs use data annotations and FluentValidation-style checks in services. Invalid enrolment states, duplicate completions, and unauthorised course access return 400/403/404 without exposing internal exceptions.

8. Sources

- Course materials: SE 410 lecture notes and project specification (Asst. Prof. Serhat Uzunbayır)

- Microsoft documentation: ASP.NET Core, Entity Framework Core, JWT Bearer authentication

- BCrypt.Net-Next library documentation

- GitHub repository: team project source code (submitted with this report)

- Demo video: screen recording of end-to-end flow (web login, enrolment approval on desktop, module completion, study plan)

- User manual: PDF describing installation, login, and main workflows for students and administrators

9. User Interface Design

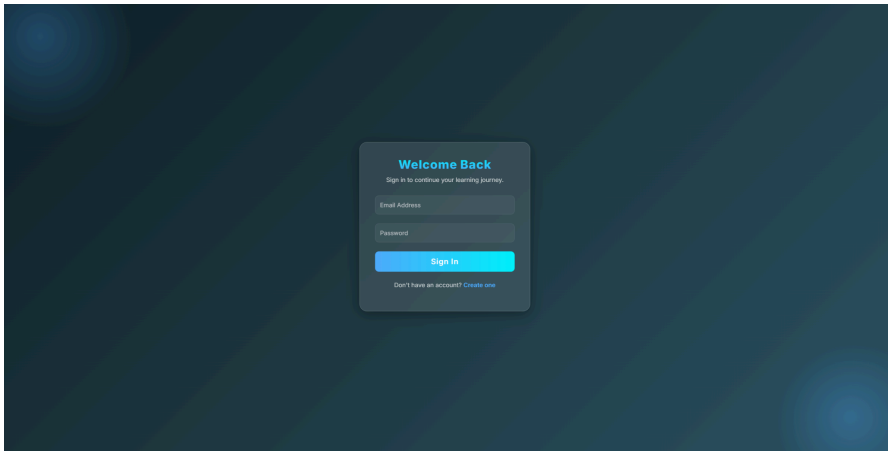
The interface follows a consistent card-based layout with clear primary actions and colour-coded feedback.

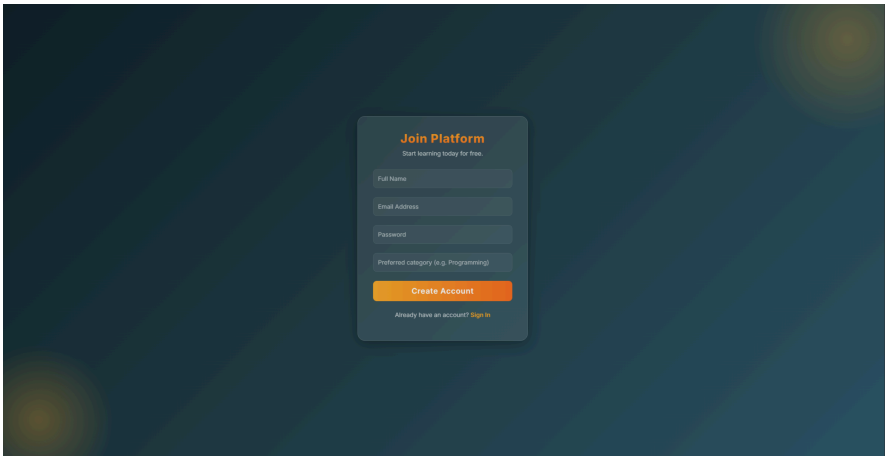
9.1. Web application (MVC)

Key screens:

- Login and 2FA — centred form, validation messages, redirect to dashboard
- Course catalogue — filter bar (category, difficulty), course cards with ECTS and enrol button
- Student dashboard — progress bars per enrolled course, link to modules and study plan
- Module progress — checklist of modules with Mark complete action
- Study plan — weekly grid showing recommended hours per course

Design choices: Bootstrap-based responsive layout, shared _Layout.cshtml navigation, success/error TempData alerts after POST actions. Screenshots are included in the submission appendix (Figures 1–4).





learning_platform Dashboard Courses Study Plan Can Ozer

Welcome back, Can Ozer!

Sign Out

Enrolled Courses
2

Completed Courses
1

Overall Progress
50,0%

Your Active Courses

C# ile Nesne Yönelimli Programlama
Modules completed: 0 / 0
Continue Course →

Müzik ve Bilgisayarlar
Modules completed: 3 / 3
Continue Course →

© 2026 - learning_platform - Privacy

learning_platform Dashboard Courses Study Plan Can Ozer

← Back to Dashboard

Müzik ve Bilgisayarlar
You have completed 3 out of 3 modules.
100,0%

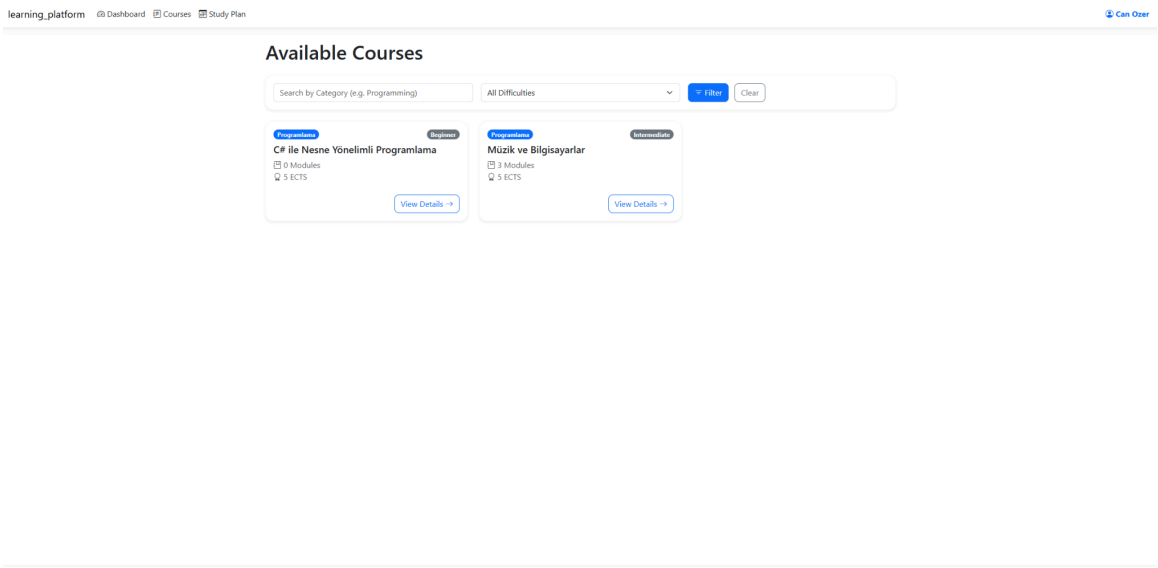
Course Modules

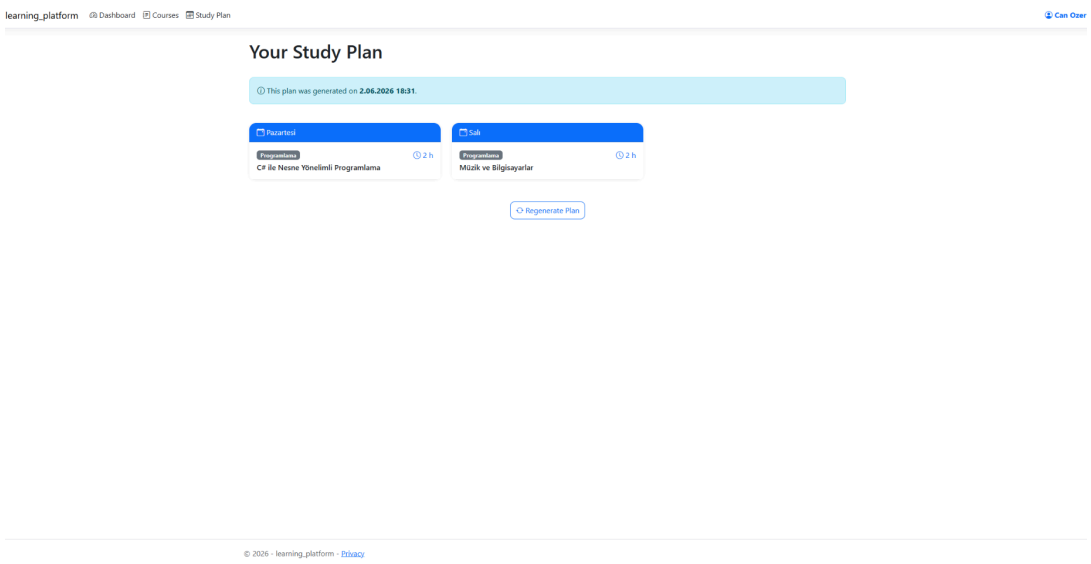
Module 1: Introduction to Digital Audio
Completed on 2.06.2026 18:11
Completed

Module 2: MIDI and Sequencing
Completed on 2.06.2026 18:11
Completed

Module 3: Algorithmic Composition
Completed on 2.06.2026 18:11
Completed

© 2026 - learning_platform - Privacy





9.2. Desktop application (WinForms)

- Login form — same credentials as web; stores JWT for subsequent API calls

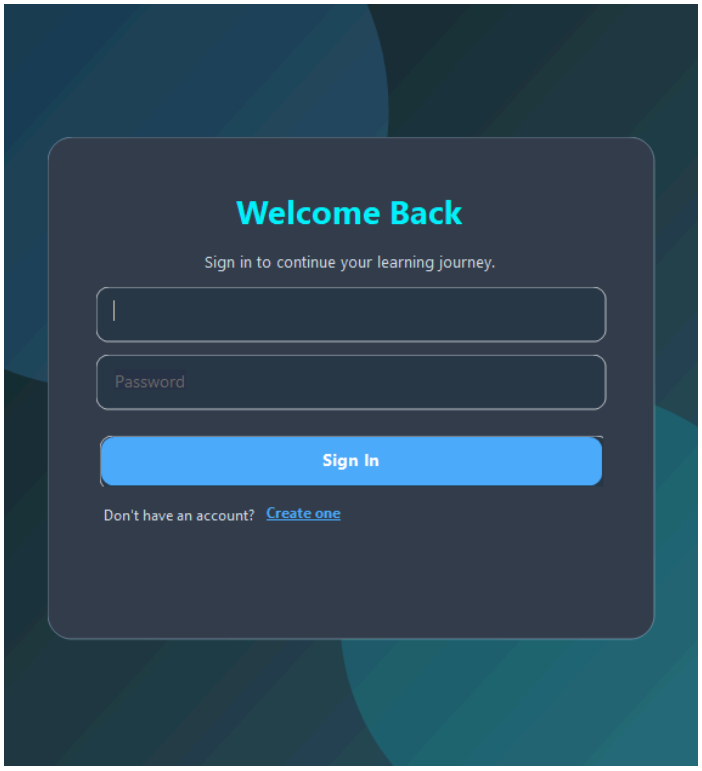
- Admin dashboard — summary cards for total students, active enrolments, average completion

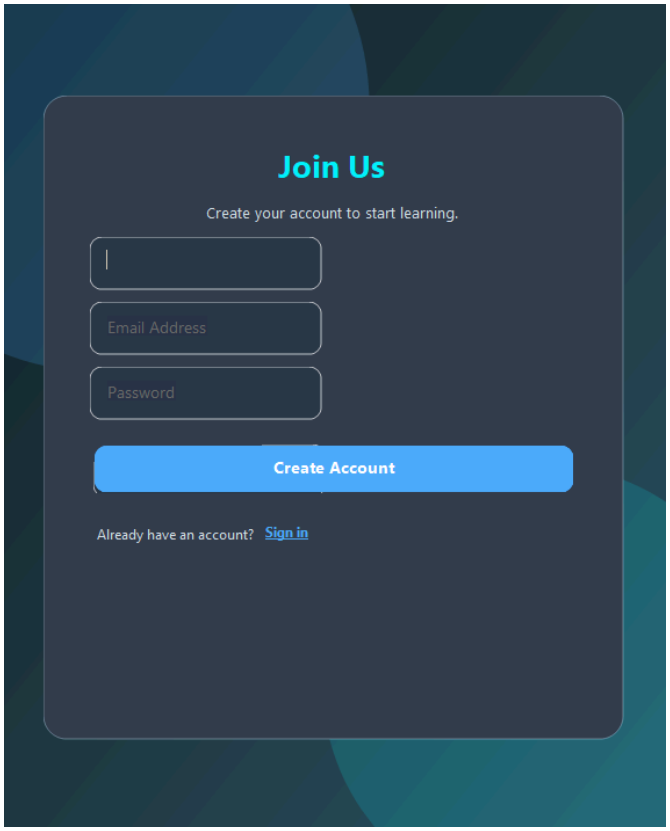
- Pending enrolments — DataGridView with Approve/Reject buttons

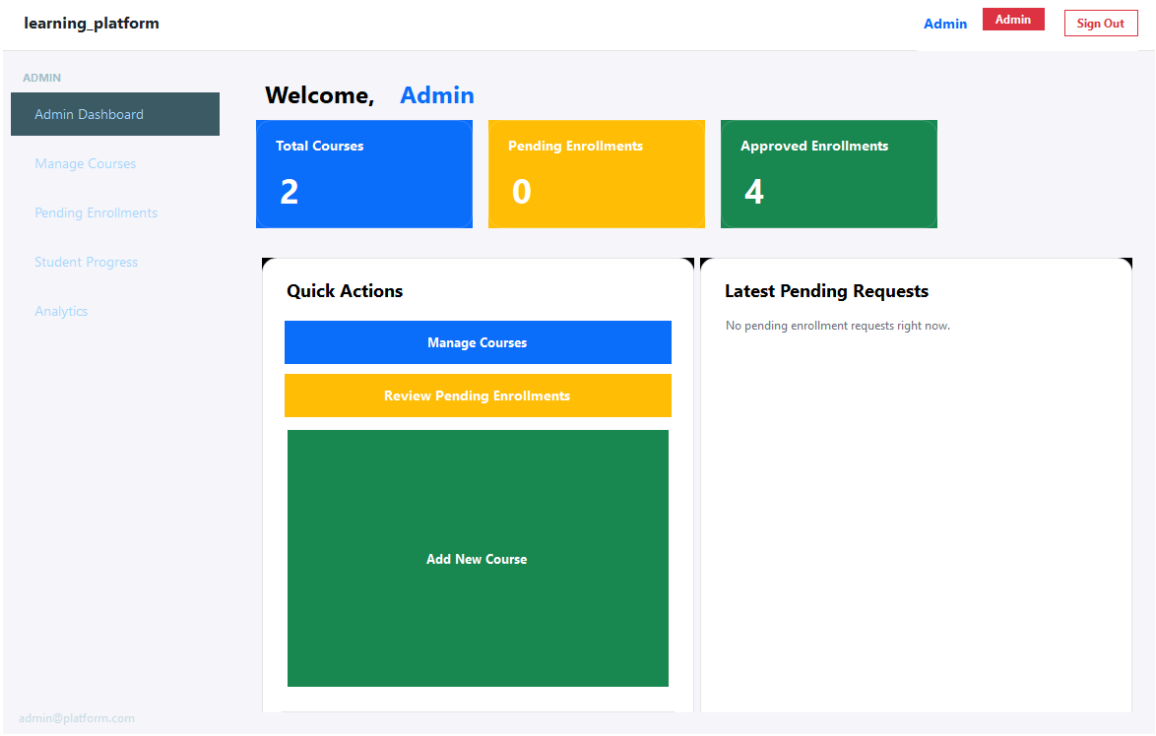
- Course and module management — tabbed editor for CRUD operations

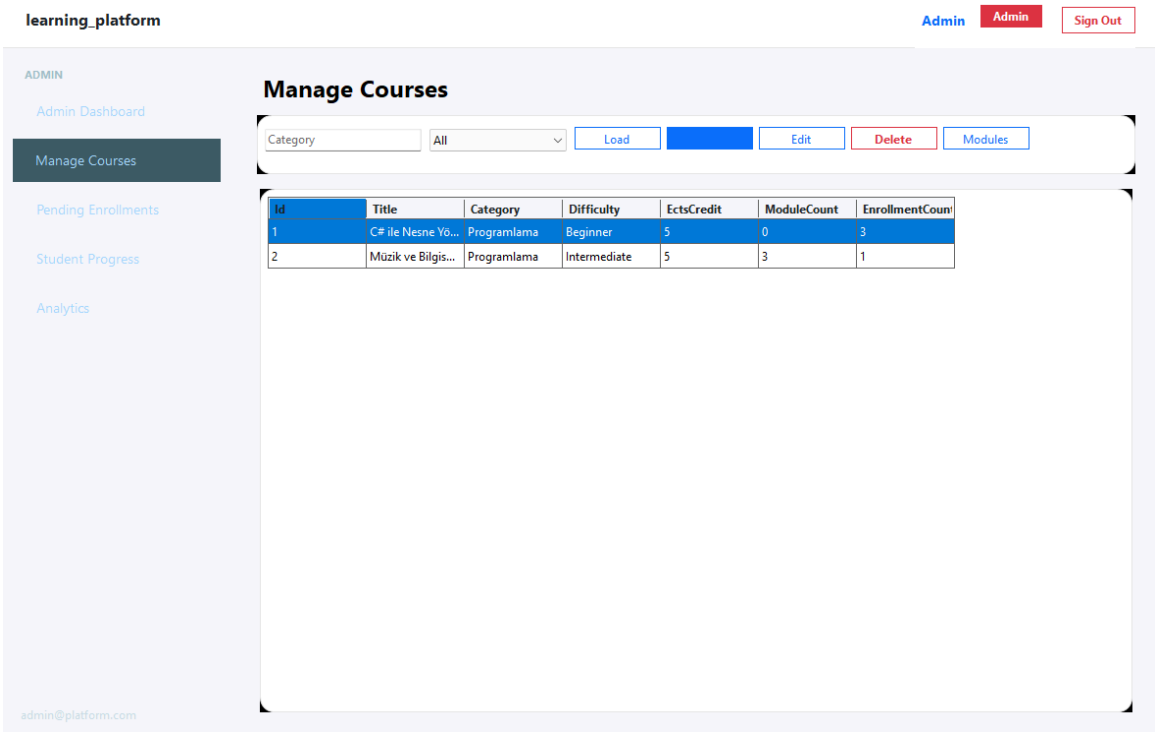
- Student progress view — filter by course, read-only completion grid

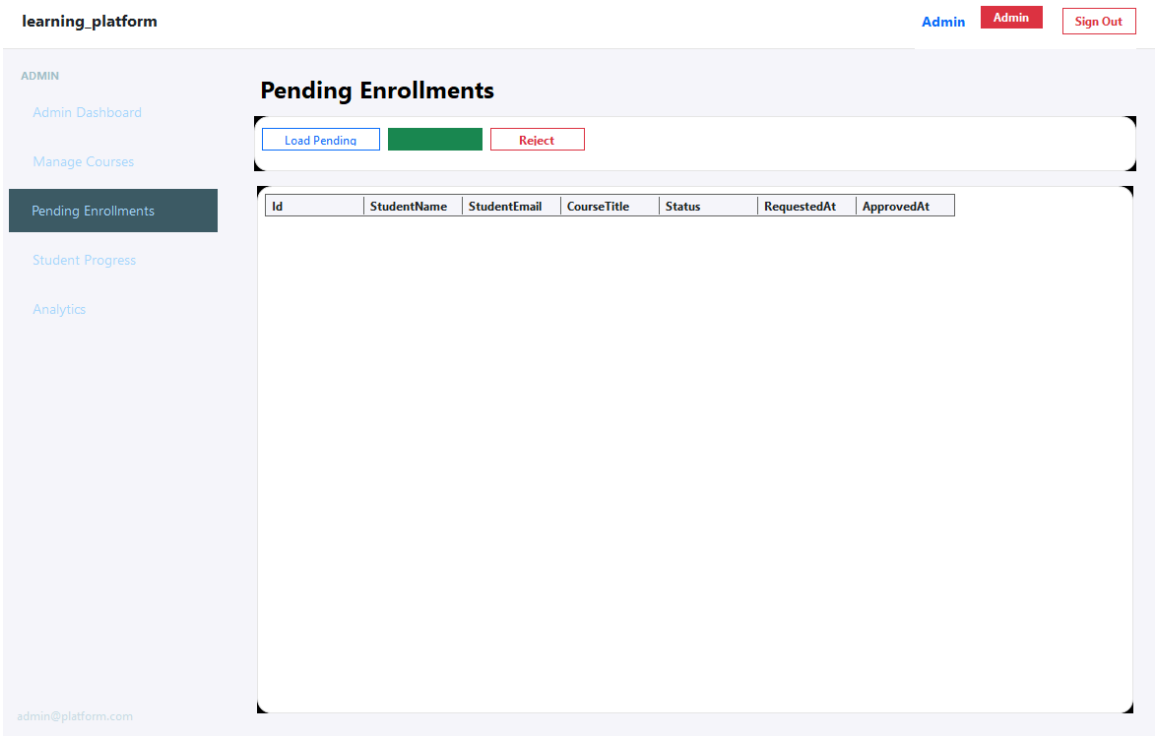
The desktop shell uses a dark sidebar for navigation and light content panels, matching the web colour palette for a unified brand (Figures 5–7 in appendix).

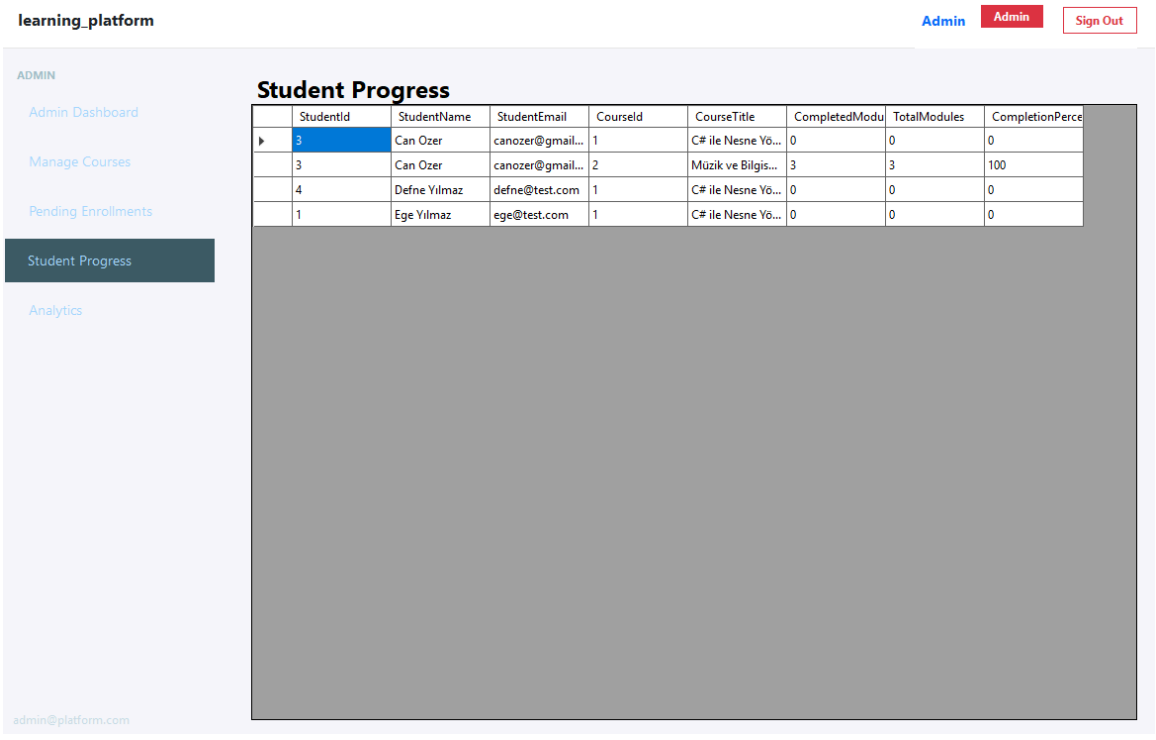

Welcome Back
Sign in to continue your learning journey.
Password
Sign In
Don't have an account? Create one

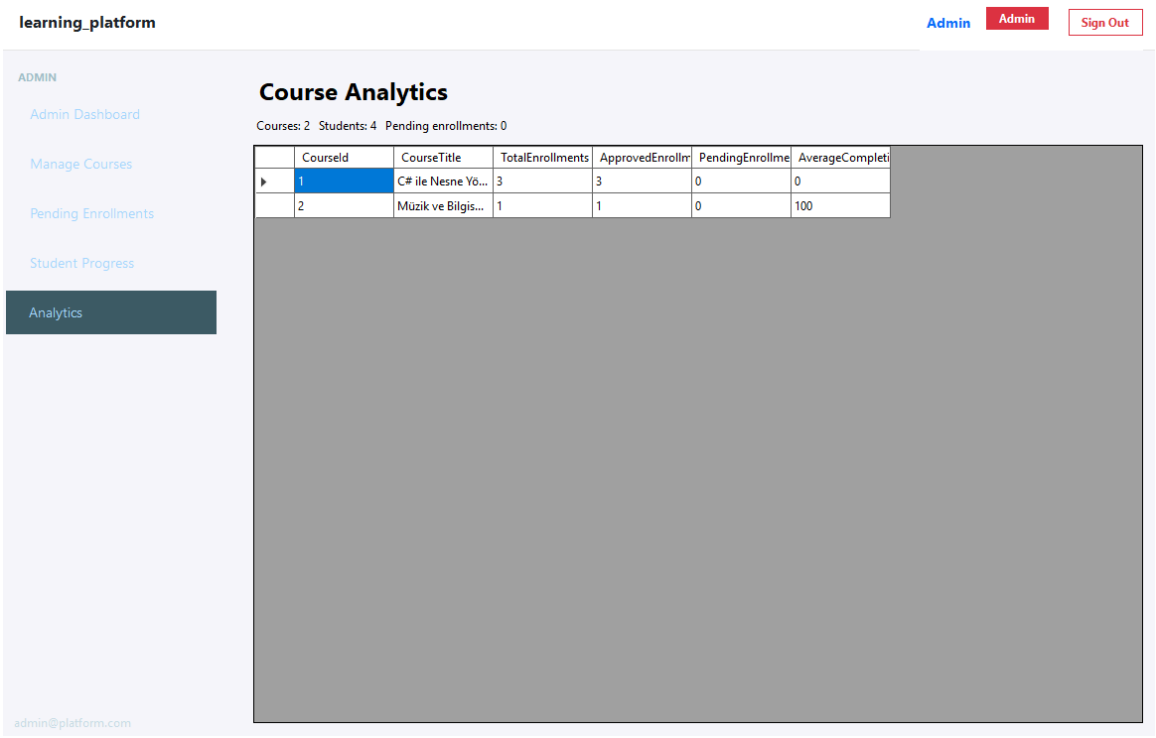

Join Us
Create your account to start learning.
Email Address
Password
Create Account
Already have an account? Sign in


learning_platform
Admin
Admin
Sign Out
ADMIN
Admin Dashboard
Manage Courses
Pending Enrollments
Student Progress
Analytics
Welcome, Admin
Total Courses
2
Pending Enrollments
0
Approved Enrollments
4
Quick Actions
Manage Courses
Review Pending Enrollments
Add New Course
Latest Pending Requests
No pending enrollment requests right now.
admin@platform.com


learning_platform
Admin
Admin
Sign Out
ADMIN
Admin Dashboard
Manage Courses
Pending Enrollments
Student Progress
Analytics
Manage Courses
Category
All
Load
Edit
Delete
Modules
Id
Title
Category
Difficulty
EctsCredit
ModuleCount
EnrollmentCoun
1
C# ile Nesne Yö...
Programlama
Beginner
5
0
3
2
Müzik ve Bilgis...
Programlama
Intermediate
5
3
1
admin@platform.com


learning_platform
Admin
Admin
Sign Out
ADMIN
Admin Dashboard
Manage Courses
Pending Enrollments
Student Progress
Analytics
admin@platform.com
Pending Enrollments
Load Pending
Reject
Id
StudentName
StudentEmail
CourseTitle
Status
RequestedAt
ApprovedAt


learning_platform
Admin
Admin
Sign Out
ADMIN
Admin Dashboard
Manage Courses
Pending Enrollments
Student Progress
Analytics
admin@platform.com
Student Progress
StudentId
StudentName
StudentEmail
CourseId
CourseTitle
CompletedModu
TotalModules
CompletionPerce
3
Can Ozer
canozer@gmail...
1
C# ile Nesne Yö...
0
0
0
3
Can Ozer
canozer@gmail...
2
Müzik ve Bilgis...
3
3
100
4
Defne Yılmaz
defne@test.com
1
C# ile Nesne Yö...
0
0
0
1
Ege Yılmaz
ege@test.com
1
C# ile Nesne Yö...
0
0
0



10. Tests

LearningPlatform.API.Tests contains 11 xUnit integration tests using WebApplicationFactory and an in-memory or test SQLite database. All tests pass on a clean checkout.

Test summary:

Test name	Scenario	Result
RegisterStudent_ReturnsCreated	POST /auth/register creates a student account	Pass
Login_ValidCredentials_ReturnsTwoFactorRequired	Login returns 2FA step before JWT	Pass
Verify2FA_ValidCode_ReturnsJwt	Correct 6-digit code issues bearer token	Pass
CreateCourse_AsAdmin_ReturnsCreated	Admin can create a course	Pass
ListCourses_AsStudent_ReturnsCatalogue	Authenticated student sees courses	Pass
RequestEnrolment_CreatesPendingRecord	Student enrolment request is Pending	Pass

ApproveEnrolment_ChangesStatusToApproved	Admin approval updates enrolment	Pass
CompleteModule_UpdatesProgress	Marking module complete increases percentage	Pass
GenerateStudyPlan_PersistsItems	Study plan rows saved with day and hours	Pass
GetAnalytics_ReturnsEnrolmentCounts	Admin analytics endpoint returns aggregates	Pass
GetRecommendations_ReturnsCoursesInCategory	Recommendation delegate returns up to five courses	Pass

Tests run with: dotnet test LearningPlatform.API.Tests/LearningPlatform.API.Tests.csproj

11. Conclusion

The E-Learning Platform with Personalized Study Plans successfully implements every functional and non-functional requirement defined for the project. The system is organised around a single ASP.NET Core Web API that owns all business logic and a SQLite database, with two independent clients — an ASP.NET Core MVC web application for students and a Windows Forms desktop application for administrators — communicating over authenticated HTTP.

Throughout the project the team applied the core framework concepts targeted by the course: LINQ for filtering and aggregation; delegates and events for enrolment, study-plan, and recommendation notifications; the MVC pattern for the web layer; Entity Framework Core with code-first migrations; and a layered security model based on password hashing, JWT authorization, and two-factor authentication.

Compared with version 1, version 2 completes the remaining requirements: the full desktop administration client, two-factor authentication, admin analytics, course-module management, delegate-based course recommendations, portable database configuration, and an automated test suite of 11 passing integration tests. Together, these additions turn the prototype into a complete, secure, and well-tested learning platform.

Possible future work includes delivering 2FA codes through a real email/SMS provider, expanding automated test coverage to edge cases in enrolment rejection, and adding richer analytics visualisation charts on the desktop dashboard.